arXiv:2405.13817v1 [cs.LG] 22 May 2024

Thermodynamic Natural Gradient Descent

Kaelan Donatella[*,†]
Normal Computing

Samuel Duffield[†]
Normal Computing

Maxwell Aifer
Normal Computing

Denis Melanson
Normal Computing

Gavin Crooks
Normal Computing

Patrick J. Coles
Normal Computing

Abstract

Second-order training methods have better convergence properties than gradient descent but are rarely used in practice for large-scale training due to their computational overhead. This can be viewed as a hardware limitation (imposed by digital computers). Here we show that natural gradient descent (NGD), a second-order method, can have a similar computational complexity per iteration to a first-order method, when employing appropriate hardware. We present a new hybrid digital-analog algorithm for training neural networks that is equivalent to NGD in a certain parameter regime but avoids prohibitively costly linear system solves. Our algorithm exploits the thermodynamic properties of an analog system at equilibrium, and hence requires an analog thermodynamic computer. The training occurs in a hybrid digital-analog loop, where the gradient and Fisher information matrix (or any other positive semi-definite curvature matrix) are calculated at given time intervals while the analog dynamics take place. We numerically demonstrate the superiority of this approach over state-of-the-art digital first- and second-order training methods on classification tasks and language model fine-tuning tasks.

1 Introduction

With the rise of more sophisticated AI models, the cost of training them is exploding, as world-leading models now cost hundreds of millions of dollars to train. This issue is compounded by the ending of both Moore's Law and Dennard's Law for digital hardware [20], which impacts both the runtime and energy efficiency of such hardware. This highlights a need and an opportunity for specialized, unconventional hardware targeted at improving the efficiency of training AI models.

Moreover, conventional digital hardware can be viewed as limiting the range of training algorithms that a user may consider. Researchers are missing an opportunity to co-design novel optimizers to exploit novel hardware developments. Instead, relatively simplistic optimizers, such as stochastic gradient descent (SGD), Adam [22], and their variants [27], are among the most popular methods for training deep neural networks (DNNs) and other large AI models. More sophisticated optimizers are rarely used due to the associated computational overhead on digital hardware.

A clear example of this is second-order methods, which capture curvature information of the loss landscape. These methods, while theoretically more powerful in terms of convergence properties, remain computationally expensive and harder to use, blocking their adoption. For example, natural gradient descent (NGD) [4, 30] involves calculating estimates of second-order quantities such as the Fisher information matrix and performing a costly linear system solve at every epoch. Some approximations to NGD, such as the Kronecker-factored approximate curvature (K-FAC) [31], have

[*]Correspondence to: kaelan@normalcomputing.ai
[†]These authors contributed equally to this work.

Preprint. Under review.

shown promise, and K-FAC has shown superior performance to Adam [25, 14]. However, applying such methods to arbitrary neural network architectures remains difficult [36].

In this article, we present thermodynamic natural gradient descent (TNGD), a new method to perform second-order optimization. This method involves a hybrid digital-analog loop, where a GPU communicates with an analog thermodynamic computer. A nice feature of this paradigm is flexibility: the user provides their model architecture and the analog computer serves only to accelerate the training process. This is in contrast to many proposals to accelerate the inference workload of AI models with analog computing, where the model is hardwired into the hardware, and users are unable to change the model architecture as they seamlessly would by using their preferred software tools [21, 6, 11, 1].

The analog computer in TNGD uses thermodynamic processes as a computational resource. Such thermodynamic devices have previously been proposed [10, 18, 15, 9, 26], have been theorized to exhibit runtime and energy efficiency gains [2, 13], and have been successfully prototyped [34, 3]. Our TNGD algorithm represents an instance of algorithmic co-design, where we propose a novel optimizer to take advantage of a novel hardware paradigm. TNGD exploits a physical Ornstein–Uhlenbeck process to implement the parameter update rule in NGD. It has a runtime per iteration scaling linearly in the number of parameters, and when properly parallelized it can be close to the runtime of first-order optimizers such as Adam and SGD. Hence, it is theoretically possible to achieve the computational efficiency of a first-order training method while still accounting for the curvature of the loss landscape with a second-order method. Moreover, our numerics show the competitiveness of TNGD with first-order methods for classification and extractive question-answering tasks.

2 Related work

There is a large body of theoretical research on natural gradient descent [4, 30, 7] arguing that NGD requires fewer iterations than SGD to converge to the same value of the loss in specific settings. While less is known about the theoretical convergence rate of Adam, there exists a large body of empirical evidence that NGD can converge in fewer iterations than Adam [33, 31, 32, 14, 38, 16].

However, a single iteration of NGD is generally more computationally expensive than that of SGD or Adam, which have a per-iteration cost scaling linearly in the number of parameters N. NGD typically has a superlinear (assuming the condition number scales as $\kappa = N^\alpha, \alpha > 0$ for NGD-CG) complexity in the number of parameters (although this may be reduced to linear scaling at the expense of higher-order scaling with batch size and output dimension, see Section 3). K-FAC [31] aims to reduce this complexity and invokes a block-wise approximation of the curvature matrix, which may not always hold. While first introduced for multi-layer perceptrons, K-FAC has been applied to more complex architectures, such as recurrent neural networks [32] and transformers [14], where additional approximations have to be made and where the associated computational overhead can vary.

There has been significant effort and progress towards reducing the time- and space- complexity of operations used in the inference workload of AI models, e.g., a variety of “linear attention" blocks have been proposed [40, 19, 42]. However, there has been less focus on reducing the complexity of training methods. While various approaches are taken to accelerating training using novel hardware, these efforts typically aim at reducing the constant coefficients appearing in the time cost of computation. Especially relevant to our work, analog computing devices have been proposed to achieve reduced time and energy costs of training relative to available digital technology [21, 6, 11, 1]. These devices are generally limited to training a neural network that has a specific architecture (corresponding to the structure of the analog device). To our knowledge, there has not yet been a proposal that leverages analog hardware to reduce the complexity of training algorithms such as NGD.

Given the existing results implying that fewer iterations are needed for NGD relative to other commonly used optimizers, we focus on reducing the per-iteration computational cost of NGD using a hybrid analog-digital algorithm to perform each parameter update. Our algorithm therefore demonstrates that complexity can be improved in training (not only in inference), and moreover that the per-iteration complexity of NGD can be made similar to that of a first-order training method.

3 Natural gradient descent

Let us consider a supervised learning setting, where the goal is to minimize an objective function defined as:

$$\ell(\theta) = \frac{1}{|\mathcal{D}|} \sum_{(x,y)\in\mathcal{D}} L(y, f_\theta(x)), \tag{1}$$

where $L(y, f_\theta(x)) \in \mathbb{R}$ is a loss function, $f_\theta(x)$ is the forward function that is parametrized by $\theta \in \mathbb{R}^N$. These functions depend on input data and labels $(x, y) \in \mathcal{D}$, with $\mathcal{D}$ a given training dataset. Viewed through the lens of statistics, minimizing the objective function is analogous to minimizing the Kullback-Leibler (KL) divergence from the target joint distribution $q(x, y)$ to the learned distribution $p(x, y|\theta)$ [30]. A straightforward way to optimize $\ell(\theta)$ is to follow the direction of steepest descent, defined by the negative gradient $-\nabla\ell$, defined as:

$$\frac{-\nabla\ell}{||\nabla\ell||} = \lim_{\epsilon\to 0} \frac{1}{\epsilon} \underset{d:||d||\leq\epsilon}{\arg\min}\, \ell(\theta + d), \tag{2}$$

with $||\cdot||$ the Euclidean norm. The natural gradient, on the other hand can be defined as the direction of steepest descent with respect to the KL divergence defined as:

$$\mathrm{KL}(p(x, y|\theta + d)||p(x, y|\theta)) = \iint p(x, y|\theta + d)\ \log\left(\frac{p(x, y|\theta + d)}{p(x, y|\theta)}\right)\ \mathrm{d}x\mathrm{d}y \tag{3}$$

[5]. One may then Taylor-expand this divergence as

$$\mathrm{KL}(p(x, y|\theta + d)||p(x, y|\theta)) = \frac{1}{2} d^\top F d + O(d^3), \tag{4}$$

where F is the *Fisher information matrix* [30] (or the *Fisher*), defined as:

$$F = \mathbb{E}_{p(x,y|\theta)}[\nabla \log p(x, y|\theta) \nabla \log p(x, y|\theta)^\top]. \tag{5}$$

The natural gradient is then simply defined as

$$\tilde{g} = F^{-1}\nabla\ell(\theta). \tag{6}$$

For the NGD optimizer, the update rule is then given by:

$$\theta_{k+1} = \theta_k - \eta F^{-1}\nabla\ell, \tag{7}$$

with η a learning rate. In practice, computing the Fisher information is not always feasible because one must have access to the density $p(x, y|\theta)$. A quantity that is always possible (and relatively cheap) to compute thanks to auto-differentiation is the empirical Fisher information matrix, defined as:

$$\bar{F} = JJ^\top = \frac{1}{b} \sum_{(x,y)\in\mathcal{S}} \nabla \log p(y|x, \theta) \nabla \log p(y|x, \theta)^\top, \tag{8}$$

where $\log p(y \mid x, \theta) = -L(y, f_\theta(x))$, $|\mathcal{S}| = b$ is the batch size and $\mathcal{S} \subset \mathcal{D}$. The Jacobian matrix J is defined as

$$J = \frac{1}{\sqrt{b}}[\nabla \log p(y_1|x_1, \theta), \nabla \log p(y_2|x_2, \theta), \ldots, \nabla \log p(y_b|x_b, \theta)].$$

Note that the squared gradient appearing in the second moment estimate of the Adam optimizer [22] is the diagonal of the empirical Fisher matrix. Another approximation to the Fisher matrix is the generalized Gauss-Newton (GGN) matrix, defined as:

$$G = J_f H_L J_f^\top = \frac{1}{b} \sum_{(x,y)\in\mathcal{S}} J_f^{(x,y)} H_L^{(x,y)} J_f^{(x,y)\top}, \tag{9}$$

where $J_f^{(x,y)}$ is the Jacobian of $f_\theta(x)$ with respect to θ and $H_L^{(x,y)}$ is the Hessian of $L(y, z)$ with respect to θ evaluated at $z = f_\theta(x)$. J_f is a $bd_z \times N$ matrix, and H_L is a $bd_z \times bd_z$ matrix, where d_z is the output dimension of $z = f_\theta(x)$ and N is the number of parameters (N also depends on d_z, where for deep networks it is a weak dependence).

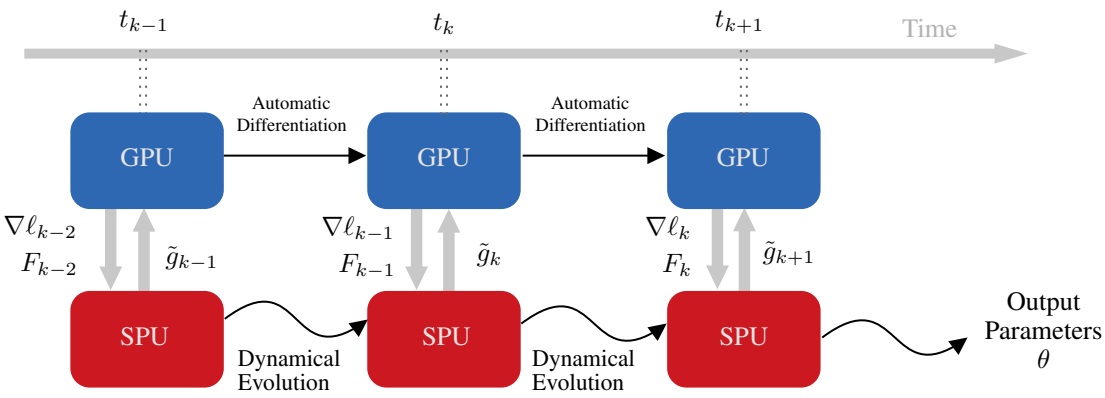

Figure 1: **Overview of Thermodynamic Natural Gradient Descent (TNGD).** A GPU that stores the model architecture and provides the gradient $\nabla \ell_k$ and Fisher matrix F_k (through its representation given by the Jacobian J_f and Hessian H_L matrices given by Eq. (9)) at step k is connected to a thermodynamic computer, called the stochastic processing unit (SPU). At times t_k, the estimate of the natural gradient $\tilde{g}_k$ is sent to the GPU, which updates the parameters of the model and calculates gradients and curvature matrices for some new data batch (x_k, y_k). During digital auto-differentiation, the SPU undergoes dynamical evolution, either continuing to approach its steady-state or remaining in it. After some time, gradient $\nabla \ell_k$ and Fisher matrix F_k are sent to the SPU through a DAC and digital controllers. This modifies the dynamics of the SPU, and after some time interval, a new natural gradient estimate $\tilde{g}_{k+1}$ is sent back to the GPU. Note that the time between two measurements $t_{k+1} - t_k$ need not be greater than the time between two auto-differentiation calls. The hybrid digital-thermodynamic process may be used asynchronously as shown in the diagram (where the time of measurement of $\tilde{g}$ and upload of the gradient and Fisher matrix are not the same).

For loss functions of the exponential family (with natural parameter z), the GGN matches the true Fisher matrix [30]. In addition, we have observed better convergence with the GGN than with the empirical Fisher (as in other works such as Refs. [33, 23], where better convergence than with the Hessian is also observed). Therefore, we will consider the GGN in what follows. Note that the methods we introduce in this work apply to any second-order optimization algorithm with a positive semi-definite curvature matrix (by curvature matrix, we mean any matrix capturing information about the loss landscape). In particular, it applies most efficiently to matrices constructed as outer products of rectangular matrices (such as the empirical Fisher and the GGN) as explained below.

3.1 Fast matrix vector products

The linear system appearing in Eq. (6) can be solved using the conjugate gradient (CG) method [33], which will be referred to as NGD-CG in what follows. In fact, when ℓ is parametrized by a neural network, the GGN-vector product Gv involved in the conjugate gradient algorithm may be evaluated in runtime $O(bN)$ thanks to fast Jacobian-vector products [8] (JVPs). This approach also enables one to not explicitly construct the Fisher matrix, thus also avoiding a $O(bd_zN^2)$ runtime cost in computing it and a $O(N^2)$ memory cost in storing it. The efficiency of this approach depends on the number of CG iterations required to obtain good performance. Importantly, convergence in $\sqrt{\kappa}$ steps, with κ the condition number of F, is not required to obtain competitive performance [31, 16]. Crucially, due to the sequential nature of the algorithm, the CG iterations cannot be parallelized.

In practice, since reaching convergence is computationally expensive, one generally stops the CG algorithm after a set number of iterations. Because of the way the step size is adapted in CG, we have observed that the solution after k steps x_k is not necessarily closer to the true solution than the initial guess x_0, in particular for ill-conditioned problems, which can make NGD-CG difficult to use.

3.2 NGD with the Woodbury identity

In the machine learning setting, it is often the case that $b \ll N$ (and $d_z \ll N$). This means that the curvature matrix is low-rank and the linear system to solve is underdetermined. To mitigate this issue,

the Fisher matrix may be dampened as $F + \lambda\mathbb{I}$. In that case, the Woodbury identity may be used to obtain the inverse Fisher vector-product $F^{-1}v$ appearing in the NGD update. We have:

$$F = UV + \lambda\mathbb{I}, \text{ with } U = J_f, V = H_L J_f^\top \tag{10}$$

$$F^{-1} = \lambda^{-1}\mathbb{I} - \lambda^{-2}U(\mathbb{I} + \lambda^{-1}VU)^{-1}V \qquad \text{(Woodbury)} \tag{11}$$

$$F^{-1}v = \lambda^{-1}\mathbb{I} - \lambda^{-2}U(\mathbb{I} + \lambda^{-1}VU)^{-1}Vv \tag{12}$$

This is included in Ref. [38], and can be competitive with NGD-CG when the batch size b and output dimension d_z are much smaller than the number of trainable parameters N. Here one must construct the V matrix, which has runtime $O(d_z^2 b^2 N)$, and invert $(\mathbb{I} + \lambda^{-1}VU)$ which is $O(b^3 d_z^3)$. While the batch size typically remains small, the value of d_z can make this inversion intractable. For example, in many language-model tasks, $d_z \sim O(10^4)$ is the vocabulary size.

4 Thermodynamic NGD

At a high level, TNGD combines the strength of GPUs (through auto-differentiation) with the strength of thermodynamic devices at solving linear systems. Regarding the latter, Ref. [2] showed that a thermodynamic device, called a stochastic processing unit (SPU), can solve a linear system $Ax = b$ with reduced computational complexity relative to standard digital hardware. The solution to the linear system is found by letting the SPU evolve under an Ornstein–Uhlenbeck (OU) process given by the following stochastic differential equation (SDE):

$$dx = -(Ax - b)dt + \mathcal{N}\left[0, 2\beta^{-1}\, dt\right], \tag{13}$$

where A is a positive matrix and β is a positive scalar (which can be seen as the inverse temperature of the noise). Operationally, one lets the SPU settle to its equilibrium state under the dynamics of Eq. (13), at which point x is distributed according to the Boltzmann distribution given by:

$$x \sim \mathcal{N}[A^{-1}b, \beta^{-1}A^{-1}]. \tag{14}$$

One can see that the first moment of this distribution is the solution to the linear system $Ax = b$. Exploiting this approach, TNGD involves a subroutine that estimates the solution to the linear system in Eq. (6). For this particular linear system, the SDE in Eq. (13) becomes the following:

$$d\tilde{g}_{k,t} = -(F_{k-1}\tilde{g}_{k,t} - \nabla\ell_{k-1})dt + \mathcal{N}\left[0, 2\kappa_0 dt\right] \tag{15}$$

$$= -(J_{f,k-1}^\top H_{L,k-1} J_{f,k-1}\tilde{g}_{k,t} - \nabla\ell_{k-1})dt + \mathcal{N}\left[0, 2\kappa_0 dt\right] \tag{16}$$

with $\tilde{g}_{k,t}$ the value of the natural gradient estimate at time t and κ_0 the variance of the noise. Comparing Eqs. (13) and (15), we see that in the equilibrium state (i.e. for large t), the mean of $\tilde{g}_{k,t}$ provides an estimate of the natural gradient, in other words:

$$\tilde{g}_k := \lim_{t\to\infty} \langle \tilde{g}_{k,t} \rangle = F_{k-1}^{-1}\nabla\ell_{k-1}. \tag{17}$$

The overall TNGD algorithm is illustrated in Fig. 1. Using the current parameter estimates θ_k, the GPU computes the matrices J_f and H_L, and the vector $\nabla\ell$, which can be accomplished efficiently using auto-differentiation. The matrices J_f, $J_f^\top$, and H_L, as well as the vector $\nabla\ell$, are uploaded to component values (see Appendix) on the SPU, which is then allowed to equilibrate under the dynamics of Eq. (15). Next, samples are taken of $\tilde{g}_{t,k}$, and are sent from the SPU to the GPU, where samples are averaged to yield an estimate of $\tilde{g}_k$. Finally, the parameters are updated using the equation

$$\theta_{k+1} = \theta_k - \eta\tilde{g}_k, \tag{18}$$

and this process may be repeated until sufficient convergence is achieved (other update equations may also be employed, see Section 5).

While Eq. (17) involves a long time limit, numerical evidence (see section 5) shows that samples may be taken even before equilibrium has been reached without harming performance significantly. Thus, the analog dynamics time t is an important hyperparameter of TNGD. Furthermore, another hyperparameter arises from the delay time t_d, defined as the time between a measurement of θ_k and the update of the gradient and GGN on the device. As discussed in Section 5, a non-zero delay time is not necessarily detrimental to performance and can in fact improve it.

Optimizer	Runtime	Memory	Model calls
SGD/Adam	$O(bN)$	$O(N)$	1
NGD	$O(N^3 + bd_zN^2)$	$O(N^2)$	bd_z
NGD-CG	$O(cbN)$	$O(N)$	$2c$
NGD-Woodbury	$O(b^2d_z^2N + b^3d_z^3)$	$O(bd_zN + b^2d_z^2)$	bd_z
Thermodynamic NGD	$O(bd_zN + t)$	$O(bd_zN + b^2d_z^2)$	bd_z

Table 1: **Runtime and memory complexity of optimizers considered in this paper.** All operations are per iteration. The first line corresponds to first-order optimizers that evaluate the gradient only, and apply diagonal rescalings and $O(N)$ operations to it only. Vanilla NGD (second line) includes the explicit storage and inversion of the GGN matrix as well as its construction, dominating the runtime and memory cost. NGD-CG (third line) can be performed by running c iterations, each dominated by GGN-vector products and has the same memory cost as first-order methods. NGD-Woodbury can be performed by constructing the matrix VU, and using the formula given by Eq. (12). This results in a runtime cost dominated by constructing VU and inverting it, which also requires its storage.

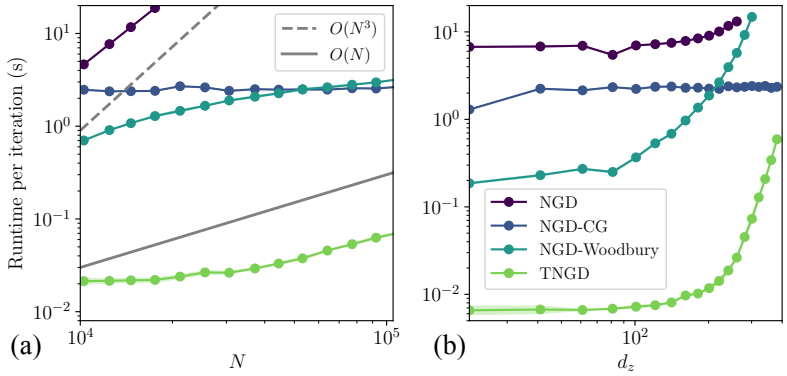

Figure 2: **Runtime per iteration of second-order optimizers considered in this paper.** (a) The runtimes per iteration are compared for NGD, NGD-CG, NGD-Woodbury, and TNGD (estimated) for various N. Here the convolutional network we applied to MNIST is used and the dimension of the hidden layer is varied to vary N for fixed $d_z = 20$. (b) The same comparison is shown for various values of d_z. The same network is used and d_z is varied (this also has the effect of varying the N). Error bars are displayed as shaded area but are smaller than the data markers.

In addition to the advantage in time- and energy-efficiency, TNGD has another advantage over NGD-CG in terms of stability. For some pathological linear systems, CG fails to converge and instead diverges. However, the thermodynamic algorithm is guaranteed to converge (on average) for any positive definite matrix. To see this, note that the mean of $\tilde{g}_{k,t}$ evolves according to

$$\langle \tilde{g}_{k,t} \rangle = \exp\left(-F_{k-1}t\right)(\tilde{g}_{k,0} - F_{k-1}^{-1}\nabla\ell_{k-1}) + F_{k-1}^{-1}\nabla\ell_{k-1}. \tag{19}$$

There is still variance associated with the estimator of $\langle \tilde{g}_{k,t} \rangle$ (the sample mean), but the sample mean converges to the solution with high probability in all cases. We also note that if we choose $\tilde{g}_{k,0} = \nabla\ell_{k-1}$, we obtain a smooth interpolation between SGD ($t = 0$) and NGD ($t = \infty$).

4.1 Computational complexity and performance

The runtime complexity of TNGD and other second-order optimization (that do not make assumptions on the structure of G, hence excluding K-FAC) algorithms is reported in Table 1. As explained, Thermodynamic NGD (TNGD) has a runtime and memory cost dominated by the construction and

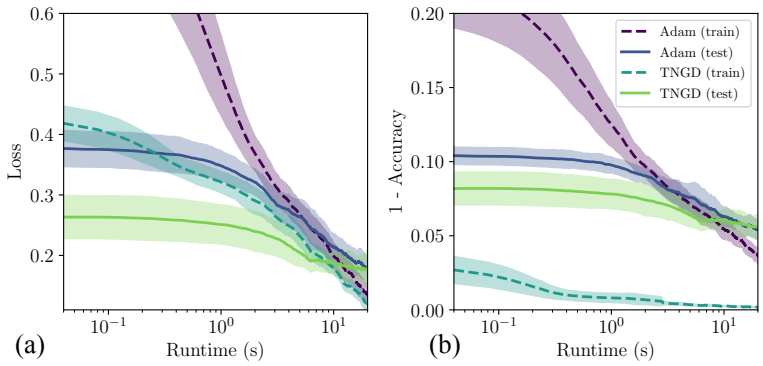

Figure 3: **Performance comparison of Adam and TNGD (simulated) on MNIST classification**. (a) Training (dashed lines) and test loss (solid lines) for Adam (darker colors) and TNGD (lighter colors) are plotted against runtime (measured for Adam, and estimated for TNGD from the timing model described in Section 4.1). Shaded areas are standard deviations over five random seeds. Note that Adam includes adaptive averaging of first and second moment estimates with $(\beta_1, \beta_2) = (0.9, 0.999)$, while TNGD does not. (b) $1 - \text{Accuracy}$ for training and test sets.

storage (before sending them off to the analog hardware) of the Jacobian of $f_\theta(x)$ and the Hessian of the loss. The t factor denotes the analog runtime, and may be interpreted similarly to c for NGD-CG as a parameter controlling the approximation. For each optimizer the number of model calls is reported. For all optimizers except NGD-CG these calls can be easily parallelized thanks to vectorizing maps in PyTorch.

In Fig. 2 a comparison of the runtime per iteration of the four second-order optimizers considered is shown. Fig. 2(a) shows the runtime as a function of the number of parameters N. The scaling of NGD as N^3 can be observed, and the NGD-CG data is close to flat, meaning the model calls parallelize well for the range of parameter count considered. The linear scaling of NGD-Woodbury and TNGD is also shown, although with a different overall behaviour due to parallelization and a much shorter runtime per iteration for TNGD. This shows that for the given range of N at $d_z = 20$, we can expect a $100\times$ speedup over second-order optimizers. Fig. 2(b) shows the dependence of runtime on the output dimension d_z for the second-order optimizers. These results indicate that TNGD is most competitive for intermediate values of d_z. Finally we note that with better hardware, the scaling with both N and d_z would be better, as the operations to construct the Hessian and Jacobian can be more efficiently parallelized for larger values.

5 Experiments

5.1 MNIST classification

We first consider the task of MNIST classification [24]. For our experiments, we use a simple convolutional neural network consisting of a convolutional layer followed by two feedforward layers, and we digitally simulate the TNGD algorithm (see App. D). The goal of these experiments is twofold: (1) to compare the estimated performance per runtime of TNGD against popular first-order optimizers such as Adam, and (2) to provide some insights on other features of TNGD, such as its performance as a function of the analog runtime t as well as its asynchronous execution as a function of the delay time t_d.

In Fig. 3(a), the training and test losses as a function of runtime for both Adam (measured) and TNGD (estimated) are presented. To estimate the TNGD runtime, we took into account results for its runtime per iteration as presented in the previous section, finding an overall $2\times$ runtime per iteration with respect to Adam for this problem on an A100 GPU. One can see from the figure that even while taking into account the difference in runtime per iteration, TNGD still outperforms Adam, especially at the initial stages of the optimization. Interestingly, it also generalizes better for the considered experimental setup. In Fig.3(b), the training and test accuracies are shown. We again see TNGD

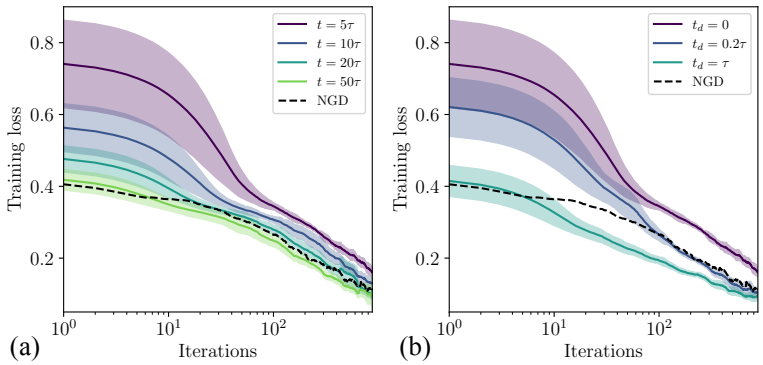

Figure 4: **Training loss vs. iterations for varying analog dynamics times**. (a) The training loss is shown for NGD (dashed line) and for TNGD with various analog dynamics times t (solid lines). (b) The training loss is shown for NGD (dashed line) and for TNGD with fixed analog dynamics time $t = 5\tau$ and varying delay times t_d (solid lines). The delay appears to have a momentum effect, which can even lead to TNGD outperforming exact NGD for certain analog dynamics and delay times. Shaded areas are standard deviations over five random seeds.

largely outperforming Adam, reaching the same training accuracy orders of magnitude faster, while also displaying a better test accuracy. These results are reminiscent of prior work on NGD [33], however here the batch size is smaller than in other works, indicating that even a noisy GGN matrix improves the optimization.

As mentioned previously, the continuous-time nature of TNGD allows one to interpolate smoothly between first- ($t = 0$) and second- ($t = \infty$) order optimization, with a given optimizer choice (whether the optimizer update rule is that of SGD or that of Adam as described in Alg. 1). In Fig. 4(a), the training loss vs. iterations is shown for various analog dynamics times. These results clearly demonstrate the effect mentioned above, where increasing the analog runtime improves performances continuously until it approaches that of exact NGD for $t \sim 50\tau$. In Fig. 4(b), the same quantity is shown for a fixed analog dynamics time t, and varying delay times t_d. This leads to a quadratic approximation of the objective function that is inaccurate (since the GGN and gradients are calculated for parameters different than the value around which the objective function is approximated). However, this results in an improved performance, even for a small delay time. A likely explanation of this result is that the state of the device retains information about the curvature of the previous quadratic approximation, while being subject to the updated quadratic approximation. This effect propagates across iterations which is reminiscent of momentum.

5.2 Language model fine-tuning

In this section we show how thermodynamic NGD may be applied to language modeling tasks, in more practically relevant settings than MNIST classification. We consider the DistilBert model [39] which we fine-tune on the Stanford question-answering dataset (SQuaD) [37], a common dataset to evaluate model comprehension of technical domains through extractive question-answering. As is commonly performed when fine-tuning, we apply a low-rank adaptation [17] to the model, which reduces its trainable parameters (details about this procedure are in App. E) to a manageable amount ($75k$ here) for limited compute resources.

Figure 5(a) displays a comparison of the training loss for different optimizers. The bare TNGD (as used in the previous section) shows a worse performance than Adam in this setting. However, a hybrid approach, TNGD-Adam, where the natural gradient estimate is used in conjunction with the Adam update rule gives the best performance (this is explained in App. B). One possible explanation for this result is that there are two pre-conditionings of the gradient for TNGD-Adam: the first comes from the natural gradient, which incorporates curvature information, and the second comes from the Adam update rule, which acts as a signal-noise ratio as explained in Ref. [22], which further adjusts the natural gradient values. In Fig. 5(b), we show that the same results as in the previous section apply to TNGD-Adam, where increasing the analog runtime boosts performance. Therefore, the

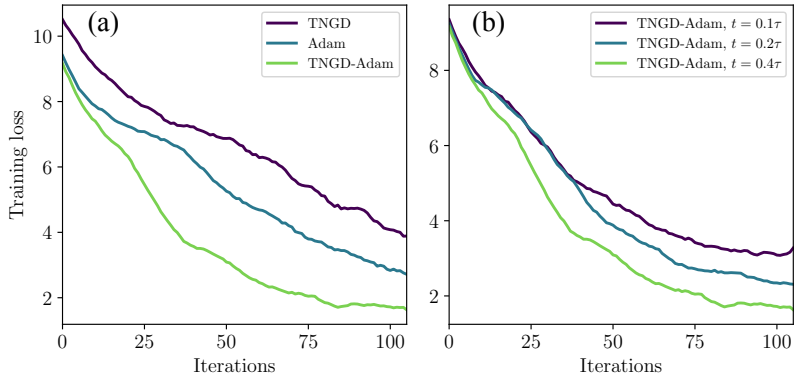

Figure 5: **Training loss vs. iterations for QA fine-tuning.** (a) Comparison of the performance per iteration of TNGD, Adam, and TNGD-Adam, where the latter uses the natural gradient estimate in conjunction with the Adam update rule with $(\beta_1, \beta_2) = (0, 0)$. (b) Performance of the TNGD-Adam optimizer for various analog dynamics times. Similar to Fig. 4, the performance improves as t grows.

analog runtime in TNGD may be viewed as a resource in this sense, that is computationally very cheap (as time constants can be engineered to be very small).

6 Limitations

The practical impact of our work relies on the future availability of analog thermodynamic computers, such as a scaled up version of the system in Ref. [34]. We provide a circuit diagram of a potential thermodynamic computer in the Appendix. Such computers can employ standard electrical components and leverage CMOS-based fabrication infrastructure, and hence are likely straightforward to scale up, although that remains to be demonstrated.

Analog computers, in general, tend to face precision issues, whereby the solution accuracy is limited by the precision of the electrical components. For analog thermodynamic computers, it is possible to mitigate this issue through an averaging technique [3], and the method proposed in Ref. [3] can be directly applied to the TNGD algorithm to improve solution accuracy. Nevertheless, we suspect that training-based applications will have a significant tolerance to precision-based errors, although a detailed study is needed to confirm that hypothesis. We note that there is a growing body of work on very low-precision inference [28] and training [41] which indicates that high numerical precision is not crucial for good performance in machine learning. We also remark that thermodynamic computers are predicted to be robust to stochastic noise sources since stochasticity is a key component of such computers [9], as is shown in Fig. 7 in the Appendix.

We have numerically tested TNGD for a small subset of potential tasks such as MNIST classification and DistilBert fine-tuning on the SQuaD dataset, for a small number of epochs. Hence, seeing if the advantage we observe for TNGD also holds for other applications is an important direction.

7 Conclusion

This work introduced Thermodynamic Natural Gradient Descent (TNGD), a hybrid digital-analog algorithm that leverages the thermodynamic properties of an analog system to efficiently perform second-order optimization. TNGD greatly reduces the computational overhead typically associated with second-order methods for arbitrary model architectures. Our numerical results on MNIST classification and language model fine-tuning tasks demonstrate that TNGD outperforms state-of-the-art first-order methods, such as Adam, and provide large speedups over other second-order optimizers. This suggests a promising future for second-order methods when integrated with specialized hardware.

Looking forward, our research stimulates further investigation into TNGD, particularly with enhancements such as averaging techniques and moving averages. Extensions to approximate second-order methods such as K-FAC may also be possible. Moreover, the principles of thermodynamic computing could inspire new algorithms for Bayesian filtering. While the current impact of our work relies on

the development and availability of large-scale analog thermodynamic computers, the theoretical and empirical advantages presented here underscore the potential of co-designing algorithms and hardware to overcome the limitations of conventional digital approaches.

References

[1] F. Aguirre, A. Sebastian, M. Le Gallo, W. Song, T. Wang, J. J. Yang, W. Lu, M.-F. Chang, D. Ielmini, Y. Yang, et al. Hardware implementation of memristor-based artificial neural networks. *Nature Communications*, 15(1):1974, 2024. URL https://www.nature.com/articles/s41467-024-45670-9.

[2] M. Aifer, K. Donatella, M. H. Gordon, T. Ahle, D. Simpson, G. E. Crooks, and P. J. Coles. Thermodynamic linear algebra. *arXiv preprint arXiv:2308.05660*, 2023. URL https://arxiv.org/abs/2308.05660.

[3] M. Aifer, D. Melanson, K. Donatella, G. Crooks, T. Ahle, and P. J. Coles. Error mitigation for thermodynamic computing, 2024. URL https://arxiv.org/abs/2401.16231.

[4] S.-I. Amari. Natural gradient works efficiently in learning. *Neural computation*, 10(2):251–276, 1998. URL http://cognet.mit.edu/journal/10.1162/089976698300017746.

[5] S.-i. Amari and H. Nagaoka. *Methods of information geometry*, volume 191. American Mathematical Soc., 2000. URL https://bookstore.ams.org/mmono-191.

[6] S. Ambrogio, P. Narayanan, H. Tsai, R. M. Shelby, I. Boybat, C. Di Nolfo, S. Sidler, M. Giordano, M. Bodini, N. C. Farinha, et al. Equivalent-accuracy accelerated neural-network training using analogue memory. *Nature*, 558(7708):60–67, 2018. URL https://www.nature.com/articles/s41586-018-0180-5.

[7] L. Bottou, F. E. Curtis, and J. Nocedal. Optimization methods for large-scale machine learning. *SIAM review*, 60(2):223–311, 2018. URL https://epubs.siam.org/doi/10.1137/16M1080173.

[8] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL http://github.com/google/jax.

[9] P. J. Coles, C. Szczepanski, D. Melanson, K. Donatella, A. J. Martinez, and F. Sbahi. Thermodynamic AI and the fluctuation frontier, 2023. URL https://arxiv.org/abs/2302.06584.

[10] T. Conte, E. DeBenedictis, N. Ganesh, T. Hylton, J. P. Strachan, R. S. Williams, A. Alemi, L. Altenberg, G. E. Crooks, J. Crutchfield, et al. Thermodynamic computing. *arXiv preprint arXiv:1911.01968*, 2019. URL https://arxiv.org/abs/1911.01968.

[11] G. Cristiano, M. Giordano, S. Ambrogio, L. P. Romero, C. Cheng, P. Narayanan, H. Tsai, R. M. Shelby, and G. W. Burr. Perspective on training fully connected networks with resistive memories: Device requirements for multiple conductances of varying significance. *Journal of Applied Physics*, 124(15), 2018. URL https://research.ibm.com/publications/perspective-on-training-fully-connected-networks-with-resistive-memories-device-requirements

[12] S. Duffield. Posteriors. https://github.com/normal-computing/posteriors, 2024.

[13] S. Duffield, M. Aifer, G. Crooks, T. Ahle, and P. J. Coles. Thermodynamic matrix exponentials and thermodynamic parallelism. *arXiv preprint arXiv:2311.12759*, 2023. URL https://arxiv.org/abs/2311.12759.

[14] R. Eschenhagen, A. Immer, R. Turner, F. Schneider, and P. Hennig. Kronecker-factored approximate curvature for modern neural network architectures. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/6a6679e3d5b9f7d5f09cdb79a5fc3fd8-Paper-Conference.pdf.

[15] N. Ganesh. A thermodynamic treatment of intelligent systems. In *2017 IEEE International Conference on Rebooting Computing (ICRC)*, pages 1–4, 2017. doi: 10.1109/ICRC.2017.8123676. URL https://ieeexplore.ieee.org/abstract/document/8123676?casa_token=wcQ38wymLlIAAAAA:Z91ud4DdK-SN8ymrUrlYCog_MM4GmdUIifsoVqpTY-kskCW0qYKlwyWTw3e_eUe19MxG5osz.

[16] M. Gargiani, A. Zanelli, M. Diehl, and F. Hutter. On the promise of the stochastic generalized gauss-newton method for training dnns. *arXiv preprint arXiv:2006.02409*, 2020. URL https://arxiv.org/abs/2006.02409.

[17] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. URL https://arxiv.org/abs/2106.09685.

[18] T. Hylton. Thermodynamic neural network. *Entropy*, 22(3):256, 2020. doi: 10.3390/e22030256. URL https://www.mdpi.com/1099-4300/22/3/256.

[19] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020. URL https://proceedings.mlr.press/v119/katharopoulos20a.html.

[20] H. N. Khan, D. A. Hounshell, and E. R. Fuchs. Science and research policy at the end of moore's law. *Nature Electronics*, 1(1):14–21, 2018. URL https://www.nature.com/articles/s41928-017-0005-9.

[21] S. Kim, T. Gokmen, H.-M. Lee, and W. E. Haensch. Analog cmos-based resistive processing unit for deep neural network training. In *2017 IEEE 60th International Midwest Symposium on Circuits and Systems (MWSCAS)*, pages 422–425. IEEE, 2017. URL https://ieeexplore.ieee.org/abstract/document/8052950.

[22] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015. URL http://arxiv.org/abs/1412.6980.

[23] F. Kunstner, P. Hennig, and L. Balles. Limitations of the empirical fisher approximation for natural gradient descent. *Advances in neural information processing systems*, 32, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/46a558d97954d0692411c861cf78ef79-Abstract.html.

[24] Y. LeCun. The mnist database of handwritten digits. *http://yann. lecun. com/exdb/mnist/*, 1998. URL http://yann.lecun.com/exdb/mnist/.

[25] W. Lin, F. Dangel, R. Eschenhagen, K. Neklyudov, A. Kristiadi, R. E. Turner, and A. Makhzani. Structured inverse-free natural gradient: Memory-efficient & numerically-stable kfac for large neural nets. *arXiv preprint arXiv:2312.05705*, 2023. URL https://arxiv.org/abs/2312.05705.

[26] P. Lipka-Bartosik, M. Perarnau-Llobet, and N. Brunner. Thermodynamic computing via autonomous quantum thermal machines. *arXiv preprint arXiv:2308.15905*, 2023. URL https://arxiv.org/abs/2308.15905.

[27] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017. URL https://arxiv.org/abs/1711.05101.

[28] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei. The era of 1-bit llms: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024. URL https://arxiv.org/abs/2402.17764.

[29] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, and B. Bossan. Peft: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022.

[30] J. Martens. New insights and perspectives on the natural gradient method. *The Journal of Machine Learning Research*, 21(1):5776–5851, 2020. URL https://arxiv.org/abs/1412.1193.

[31] J. Martens and R. Grosse. Optimizing neural networks with kronecker-factored approximate curvature. In *International conference on machine learning*, pages 2408–2417. PMLR, 2015. URL https://proceedings.mlr.press/v37/martens15.html.

[32] J. Martens, J. Ba, and M. Johnson. Kronecker-factored curvature approximations for recurrent neural networks. In *International Conference on Learning Representations*, 2018. URL https://openreview.net/pdf?id=HyMTkQZAb.

[33] J. Martens et al. Deep learning via hessian-free optimization. In *ICML*, volume 27, pages 735–742, 2010. URL https://www.cs.toronto.edu/~asamir/cifar/HFO_James.pdf.

[34] D. Melanson, M. A. Khater, M. Aifer, K. Donatella, M. H. Gordon, T. Ahle, G. Crooks, A. J. Martinez, F. Sbahi, and P. J. Coles. Thermodynamic computing system for AI applications. *arXiv preprint arXiv:2312.04836*, 2023. URL https://arxiv.org/abs/2312.04836.

[35] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[36] J. G. Pauloski, Z. Zhang, L. Huang, W. Xu, and I. T. Foster. Convolutional neural network training with distributed k-fac. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–12. IEEE, 2020. URL https://ieeexplore.ieee.org/abstract/document/9355234.

[37] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang. Squad: 100,000+ questions for machine comprehension of text. *arXiv preprint arXiv:1606.05250*, 2016. URL https://arxiv.org/abs/1606.05250.

[38] Y. Ren and D. Goldfarb. Efficient subsampled gauss-newton and natural gradient methods for training neural networks. *arXiv preprint arXiv:1906.02353*, 2019. URL https://arxiv.org/abs/1906.02353.

[39] V. Sanh, L. Debut, J. Chaumond, and T. Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019. URL https://arxiv.org/abs/1910.01108.

[40] Z. Shen, M. Zhang, H. Zhao, S. Yi, and H. Li. Efficient attention: Attention with linear complexities. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 3531–3539, 2021. URL https://ieeexplore.ieee.org/abstract/document/9423033.

[41] X. Sun, N. Wang, C.-Y. Chen, J. Ni, A. Agrawal, X. Cui, S. Venkataramani, K. El Maghraoui, V. V. Srinivasan, and K. Gopalakrishnan. Ultra-low precision 4-bit training of deep neural networks. *Advances in Neural Information Processing Systems*, 33:1796–1807, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/13b919438259814cd5be8cb45877d577-Abstract.html.

[42] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020. URL https://arxiv.org/abs/2006.04768.

A Levenberg-Marquardt regularization schedule

Poor-conditioning and singularity of the curvature matrix can greatly decrease the performance of NGD, which is dealt with by adding a term $\lambda\mathbb{I}$ to the curvature matrix. Following [33], it is possible to use a simple method to adapt the value of λ at each time step, known as a Levenberg-Marquardt schedule. This involves computing the reduction ratio ρ, defined as:

$$\rho = \frac{\ell(\theta_{k+1}) - \ell(\theta_k)}{q_{\theta_k}(\theta_{k+1} - \theta_k) - q_{\theta_k}(0)} \tag{20}$$

with $q_{\theta_k}(p)$ the quadratic approximation to ℓ around θ_k defined as:

$$q_{\theta_k}(p) = \ell(\theta_k) + \nabla\ell(\theta_k)^\top p + \frac{1}{2}p^\top G_k p. \tag{21}$$

If $\rho > a$, $\lambda \leftarrow \alpha\lambda$, and if $\rho < 1 - a$, $\lambda \leftarrow \lambda/\alpha$. This can be interpreted as distrusting the quadratic model when ρ is small, hence increasing λ for the next iteration. This procedure may be used for TNGD, however this adds a supplementary digital cost of a GGN-vector product $G_k p$ (which has a similar cost to two JVPs). For our experiments we did not find it to significantly boost performance although it may be considered for future work.

B TNGD algorithm

In Alg. 1 we provide the steps for the TNGD algorithm. This algorithm may be used in conjunction with various digital optimizers (such as SGD or Adam). The thermodynamic linear solver (TLS) is performed by an analog thermodynamic computer whose physical implementation is described in appendix C. The TLS takes as inputs the Jacobian $J_{f,k}$, the Hessian H_L, the gradient g_k and an initial point x_0 (that can be reset at each iteration, or not, in which case $t_d > 0$).

Algorithm 1 Thermodynamic Natural Gradient Descent

Require: $n > 0$
  Initialize θ_0
  $\tilde{g}_0 \leftarrow \nabla\ell(\theta_0)$
  `optimizer` $\leftarrow$ `SGD`(η, β) **or** `Adam`(η, β_1, β_2)
  while $k \neq n$ **do**
    $x_k, y_k \leftarrow$ next batch
    $g_k \leftarrow \nabla\ell(\theta_k, x_k, y_k)$
    $\tilde{g}_k \leftarrow$ `TLS`$(J_f, H_L, b = g_k, x_0 = \tilde{g}_{k-1})$
    `optimizer.update`$(\theta_k, \tilde{g}_k)$
    $k \leftarrow k + 1$
  end while

C Hardware implementation

The thermodynamic NGD algorithm can be implemented in similar hardware to what is presented in Ref. [2, 34]. However, this requires one to construct the full curvature matrix, which is quadratic in the number of parameters, and then send it to the analog hardware. Therefore, an alternative hardware implementation that is described by the same electronic circuit equations is preferred. This alternative implementation is comprised of three arrays of resistors of size (bd_z, N), (bd_z, bd_z) and (N, bd_z) for storing J_f, H_L and $J_f^\top$, respectively (hence two of these are rectangular resistor arrays). These three arrays of resistors enable one to implement the following differential equation in hardware:

$$dV = -(J_f H_L J_f^\top + \lambda\mathbb{I})V dt - b dt + \mathcal{N}(0, 2\kappa_0\sqrt{dt}) \tag{22}$$

where κ_0 is the noise variance and $V = (V_1, V_2, \ldots, V_N)$ is a vector of voltages.

To understand this let us consider a single resistor resistor array shown in Fig. 6. By Kirchhoff's current law, we obtain the equation of motion for the vector of voltages $V = (V_1, V_2, V_3)$ as:

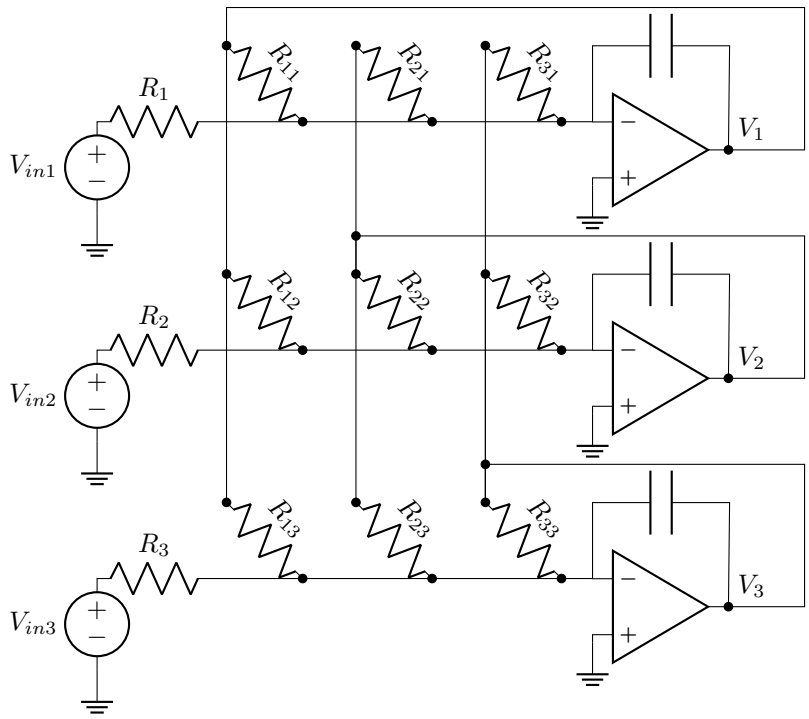

Figure 6: **Circuit diagram for the thermodynamic device in the case of a single resistor array implementation for a 3-dimensional problem.** The device is comprised of three voltage sources, each of which is connected to three resistors $\{R_i\}$. Each of these resistors is connected to a line that goes into the negative pin of a different operational amplifier. A capacitor connects the negative pin of the operational amplifier to the operational amplifier's output port, where the voltage is denoted by $\{V_i\}$. The output ports of the operational amplifiers are connected to an array of $N \times N$ resistors $\{R_{ij}\}$ (nine here), each of which connects to the line going from the resistors $\{R_i\}$ to the negative pins of the operational amplifiers. This feedback loop enables the circuit to run a differential equation whose steady-state corresponds to the solution of a linear system of equations.

$$C\dot{V} = -\mathcal{G}V + R^{-1}V_{\rm in} + I_{\rm n}, \tag{23}$$

with $V_{\rm in} = (V_{\rm in1}, V_{\rm in2}, V_{\rm in3})$, $R = {\rm diag}(R_1, R_2, R_3)$, $C = {\rm diag}(C_1, C_2, C_3)$, and $I_{\rm n} = (I_{\rm n_1}, I_{\rm n_2}, I_{\rm n_3})$, which is the current noise vector and is assumed to be Gaussian with zero mean and variance $2\kappa_0$. In this case we have

$$\mathcal{G} = \begin{pmatrix} \frac{1}{R_{11}} & \frac{1}{R_{21}} & \frac{1}{R_{31}} \\ \frac{1}{R_{12}} & \frac{1}{R_{22}} & \frac{1}{R_{32}} \\ \frac{1}{R_{13}} & \frac{1}{R_{23}} & \frac{1}{R_{33}} \end{pmatrix}. \tag{24}$$

These resistor values need to be tunable such that different Jacobians and Hessians may be given as inputs throughout the training process (and for the hardware to be used for different problems).

At steady state ($\dot{V} = 0$), the average voltage vector is $\langle V \rangle = \mathcal{G}^{-1}R^{-1}V_{\rm in}$, which corresponds to the solution of the linear system $Ax = b$ with $A = \mathcal{G}$, $x = V$, $b = R^{-1}V_{\rm in}$.

One may compose three arrays of resistors to obtain Eq. (22), giving a total resistor count of $bd_z(bd_z + 2N)$ (with two of them having no capacitors, which enables one to obtain a first-order SDE). The damping term proportional λ may be obtained by adding resistors, which has the physical effect of stabilizing the electrical system (as it does numerically).

One may run the thermodynamic linear solver by setting the voltage values $V_{\rm in}$ to the the gradient $\nabla\ell$ with a digital-to-analog converter, and set the values of the programmable resistors thanks to a digital controller.

To obtain the comparisons to other digital methods, we considered the following procedure to run the TLS on electrical hardware. In such a device the $\tau = RC$ time constant sets the characteristic timescale of the thermodynamic device.

1. Digital-to-analog (DAC) conversion of the the gradient vector with a given bit-precision.
2. Set the configuration of the programmable resistors ($bd_z(bd_z + 2N)$) values with a given bit precision to set).
3. Let the dynamics run for t (the analog dynamic time). Note that for simulations this time was chosen heuristically by exploring convergence in the solutions of the problem of interest.
4. Analog-to-digital (ADC) conversion of the solution measured at nodes V_i to the digital device.

The runtime estimated are based on the following assumptions:

- 16 bits of precision.
- A digital transfer speed of $50Gb/s$.
- $R = 10^3\,\Omega$, $C = 1$ nF, which means $RC = 1\mu$s is the characteristic timescale of the system.

Finally, note that in all cases that were investigated, the dominant contribution to the total runtime of TNGD was the digital steps to compute the gradients, Jacobian and Hessian matrices. Hence some assumptions about the DAC/ADC may be relaxed and the total TNGD runtime would be similar. The RC time constant may also be reduced to make the algorithm faster, although this is found to be easily sub-dominant with respect to input operations (setting the configuration of the device).

D Simulating TNGD

The results reported in this paper require simulating the thermodynamic device. To do so, we employ a Euler-Maruyama discretization of Eq. 15, where the update equation is:

$$\tilde{g}^{(k+1)} = \tilde{g}^{(k)} + \delta t(G\tilde{g}^{(k)} - \nabla\ell) + z\sqrt{2\kappa_0\delta t} \tag{25}$$

where δt is a step size, $z \sim \mathcal{N}(0, 1)$ and the GGN-vector product $G\tilde{g}^{(k)}$ is evaluated in linear time, with no need to construct G as explained in Section 3. One may consider higher-order schemes, which in general will cost d GGN-vector products (hence $2d + 1$ model calls, accounting for the gradient) for each step of an order d solver.

With an Euler-Maruyama scheme, one therefore requires 3 model calls per time step, which results in long simulation times for the larger t values we report.

E Experimental details

All experiments except the one reported in Fig. 2 were carried out on a Nvidia A100 GPU with 80 Gb of RAM. The experiment corresponding to Fig. 2 was carried out on a AMD EPYC 7763 64-Core CPU with 32 GB of RAM (the results on the GPU had too much variance even for a large number of repetitions). For Fig. 2, $b = 32$, $c = 200$, and the results were obtained by repeating over 5 manual random seeds, with the standard deviation over runs being shown as shaded areas. Modifying c has the simple effect of shifting the curve on the scale.

All experiments are written in PyTorch [35], and we have used the `posteriors` library [12] which supports GGN-vector products, constructing GGN matrices (for exact NGD and NGD-Woodbury) and the conjugate gradient solver in PyTorch.

E.1 MNIST

For the MNIST experiments, we train on 10,000 images and test on 10,000 other images, with $b = 64$ for 10 epochs. Below is a table with hyperparameters for each figure in the main text:

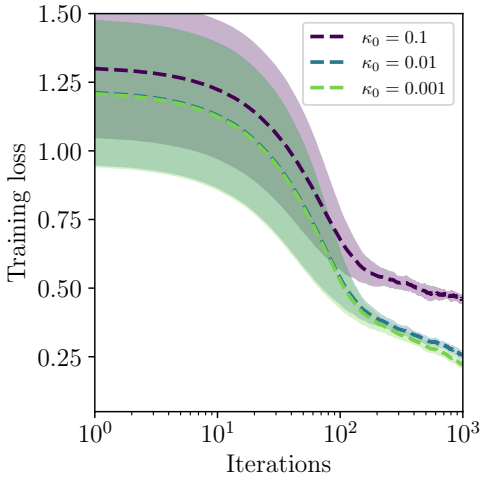

Figure 7: **Training loss vs. iterations for varying noise levels.** The noise level is defined by the noise variance κ_0 entering Eq. (15). Here $t = \tau$.

Figure – Optimizer	Optimizer parameters
Fig. 3 – Adam	$\eta = 0.001, \beta_1 = 0.9, \beta = 0.999, \epsilon = 1e-8$
Fig. 3 – TNGD	$\eta = 0.01, \beta = 0, t = 50\tau, t_d = 0, \lambda = 0.01, \delta t = 0.1\tau$
Fig. 2(a) – TNGD	$\eta = 0.01, \beta = 0, \lambda = 0.01, \delta t = 0.1\tau$

For these experiments, the plotted data is the mean (and standard deviation) of the moving average over 200 points for the five same manual random seeds. The Adam experiments took ~ 5 minutes to run, while the longest TNGD experiments took ~ 14 hours (due to many time steps being required, see Section D. We performed sweeps over the damping and learning rate value of TNGD which we do not report in the paper that took ~ 10 days of accumulated total runtime.

E.1.1 Noisy simulation

One key feature of TNGD is that it is noise-resilient. Indeed, because the solution of the linear system of Eq. (6) is encoded in the first moment of the equilibrium distribution, any noise that is approximately Gaussian will not affect much the quality of the results for reasonable noise levels. In Fig. 7, the loss vs. iterations are shown for varying noise levels, which are defined by the value of the noise variance κ_0. For $\kappa_0 < 0.01$, the noise essentially does not affect the performance of TNGD (as the influence of noise of performance starts to saturate at this value), even for a very small analog dynamics time (here, $t = \tau$). For a realistic electrical device where θ_t are voltages, the contribution of thermal noise to the noise level would be $\kappa_0 \sim 10^{-6}V$. Other noise sources may contribute to the noise level, but because of the nature of the TNGD algorithm, it exhibits a high noise-resilience. Note that it is also possible to collect more samples from the device to reduce influence of the noise if the noise level is large.

E.2 Extractive question-answering

For the QA experiments, we train on 800 articles and test on 200 other articles of the SQuaD dataset, with $b = 32$ for 5 epochs. Below is a table with hyperparameters for Fig. 5:

Figure – Optimizer	Optimizer parameters
Fig. 5(a) – TNGD	$\eta = 0.01, \beta = 0, t = 0.4\tau, t_d = 0, \lambda = 1$
Fig. 5(a) – Adam	$\eta = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 1e-8$
Fig. 5(b) – TNGD-Adam	$\eta = 0.001, \beta_1 = 0, \beta_2 = 0, \epsilon = 1e-8, t_d = 0, \lambda = 1, \delta t = 0.02\tau$

We apply low-rank adaptation (LoRA) to the Q, K, V modules and output projection matrices of the attention layers with parameters $r = 2, \alpha = 32$ and a dropout of 0.1. LoRA consists in replacing the

pre-trained weight matrices of the targeted layers W_0 by:

$$\tilde{W} = W_0 + AB, \tag{26}$$

where A and B are two rectangular matrices with their smaller dimension being α (hence AB is low-rank). We used the `peft` package [29], which interfaces smoothly with PyTorch and `posteriors`.

For these experiments we only report a single fixed random seed due to long simulation times. The Adam experiments took ~ 20 minutes, while the longest TNGD experiments took ~ 2 days (due to many time steps being required, see Section D. We performed sweeps over the damping and learning rate value of TNGD and Adam which we do not report in the paper that took ~ 10 days of accumulated total runtime.